

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 8

NLP & COMPUTER VISION — SPRINT 3

Text Processing, Image Processing, Integration & Full Model Evaluation

The third sprint of the Phase 3 capstone: applied NLP and computer vision. Interns learn text preprocessing (tokenization, TF-IDF, embeddings) and image preprocessing (OpenCV, augmentation), then integrate their model into a complete, rigorously evaluated pipeline and run the full Sprint 3 review cycle.

PHASE 3 — SPRINT 3 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 8 Overview

Week	Week 8 of 10 — Phase 3: Deep Learning & Applied Project, Sprint 3
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 15-16 combined)
Format	On-site / Remote / Hybrid — notebooks in Jupyter/Colab, committed to GitHub via feature branches
Focus	NLP fundamentals (tokenization, TF-IDF, word embeddings) and computer vision basics (OpenCV, augmentation, transfer learning); Sprint 3 integration & full evaluation of the Phase 3 capstone
Prerequisite	Weeks 1-7 completed; Sprint 2 reviewed; core model developed and beating the baseline
Mentor Supervision	Mentor Code & Notebook Review mid-sprint (Day 3) via pull request; Sprint Review + Retrospective on Day 5

2. Sprint 3 Structure

Week 8 is Sprint 3 of the Phase 3 capstone project, following the program's standard sprint cycle:

Event	Description
Sprint Planning (Day 1)	Define the Sprint 3 goal and backlog (integration & full evaluation), incorporating the Sprint 2 retrospective improvement.
Daily Stand-up (Daily)	3-minute update: what was completed, what is next, any blockers.
Mentor Code & Notebook Review (Day 3)	Mentor reviews the integrated pipeline and evaluation via GitHub pull request; structured comments on methodology.
Sprint Review (Day 5)	Demo the integrated pipeline and full evaluation. Incomplete tasks move to Sprint 4 with documented reasons.
Sprint Retrospective (Day 5)	What went well, what to improve, and one specific action for Sprint 4 (deployment & polish).

3. Week 8 Learning Objectives

- Preprocess raw text for ML: tokenization, cleaning, stop-word removal, and lemmatization.
- Represent text numerically with TF-IDF and with word embeddings, and explain the difference.
- Preprocess images with OpenCV and build an augmentation pipeline for computer vision.
- Integrate the trained model into a single, clean, end-to-end pipeline.
- Evaluate the model rigorously with the right metrics, error analysis, and SHAP explainability.
- Run the full Sprint 3 cycle: planning, stand-ups, mentor review, Sprint Review, and Retrospective.

4. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Sprint 3 planning; NLP preprocessing — tokenization, cleaning, lemmatization
Day 2	8 hrs	Text representation: TF-IDF and word embeddings (Word2Vec, GloVe)
Day 3	8 hrs	Computer vision preprocessing with OpenCV; augmentation; Mentor Code Review
Day 4	8 hrs	Model integration into an end-to-end pipeline; error analysis
Day 5	8 hrs	Full evaluation and SHAP explainability; Sprint Review & Retrospective

5. Day-by-Day Curriculum

Day 1 — Sprint 3 Planning & NLP Preprocessing — 8 hours

Learning Objectives

- Complete Sprint 3 planning and define the integration & evaluation backlog.
- Tokenize and clean raw text into a consistent form.
- Apply stop-word removal and lemmatization appropriately for the task.

Key Topics

- Sprint 3 planning: goal, backlog, carrying forward the Sprint 2 retrospective
- Why text needs preprocessing
- Tokenization (word and sub-word)
- Cleaning: lowercasing, punctuation removal, stop words
- Lemmatization vs. stemming

Lesson Content

1.1 Sprint 3 Planning

Day 1 opens with Sprint Planning: define the Sprint 3 goal — integrating the model into a complete pipeline and evaluating it rigorously — break it into backlog tasks, and carry forward the improvement from the Sprint 2 retrospective. This sprint turns a trained model into a finished, trustworthy component ready for deployment in Sprint 4.

1.2 Why Text Needs Preprocessing

A model cannot read raw text — it needs numbers. But before converting text to numbers, raw text must be cleaned and standardized, because "Good", "good", and "good!" would otherwise be treated as three different tokens. NLP preprocessing is the pipeline that turns messy human language into a clean, consistent form ready for numerical representation.

1.3 Tokenization

Tokenization splits text into units (tokens) — usually words, sometimes sub-words or characters. It is the first step of every NLP pipeline: "The movie was great" becomes ["The", "movie", "was", "great"]. Modern transformers (Week 7) use sub-word tokenization, which handles unknown words gracefully by breaking them into known pieces.

```
import nltk
from nltk.tokenize import word_tokenize
tokens = word_tokenize("The movie was great!")
# ['The', 'movie', 'was', 'great', '!']
```

1.4 Cleaning and Normalization

Standard cleaning steps make tokens consistent:

Step	What It Does	Example
Lowercasing	Removes case differences	"Good" → "good"
Removing	Strips non-word noise	"great!" → "great"

punctuation/numbers		
Stop-word removal	Drops common low-signal words	removes "the", "is", "a"
Lemmatization	Reduces words to their base form	"running", "ran" → "run"

Lemmatization (reducing to the dictionary base form) is preferred over the cruder stemming (chopping off endings), because it produces real words and preserves meaning. The goal throughout is to reduce vocabulary size while keeping the signal that matters for the task.

Preprocessing choices are task-dependent: for sentiment analysis, removing "not" (a stop word in some lists) would be a serious error — it flips meaning. Always check that cleaning preserves the signal your task needs.

Hands-On Lab: Sprint 3 Kickoff & Text Cleaning

- Step 1: Complete Sprint 3 planning and select the backlog tasks for integration and evaluation.
- Step 2: Take a raw text sample (or the project's text data) and tokenize it.
- Step 3: Apply a full cleaning pipeline: lowercase, remove punctuation, remove stop words, lemmatize.
- Step 4: Verify the cleaning preserves task-critical words (e.g. negations for sentiment) and document the choices in Markdown.

Tools Used

NLTK / spaCy • Pandas • Jupyter/Colab • Git & GitHub

Day 2 — Text Representation: TF-IDF & Embeddings — 8 hours

Learning Objectives

- Convert cleaned text to numeric vectors with TF-IDF.
- Explain word embeddings and how they capture meaning.
- Choose between TF-IDF and embeddings for a given task.

Key Topics

- From text to numbers: two families
- Bag-of-words and TF-IDF weighting
- Word embeddings: Word2Vec, GloVe, semantic geometry
- TF-IDF vs. embeddings trade-offs
- Contextual embeddings from transformers

Lesson Content

2.1 From Text to Numbers

After cleaning, text must become numeric vectors. There are two main families of approaches, and understanding the difference is central to applied NLP.

2.2 Bag-of-Words and TF-IDF

The bag-of-words approach represents a document by which words it contains and how often, ignoring order. TF-IDF (Term Frequency-Inverse Document Frequency) improves on raw counts by weighting each word: a word gets a high score if it is frequent in this document (TF) but rare across all documents (IDF). This down-weights common words like "movie" and up-weights distinctive ones like "masterpiece" — surfacing the words that actually distinguish documents.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(cleaned_texts) # sparse matrix of TF-IDF scores
```

TF-IDF is simple, fast, and a strong baseline for text classification — often the right first choice for the Sentiment Analysis project, exactly as the outline suggests. Its limitation: it treats each word independently and ignores meaning and order.

2.3 Word Embeddings

Word embeddings represent each word as a dense vector positioned so that words with similar meanings sit close together in vector space. Unlike TF-IDF, embeddings capture semantic relationships: "king" and "queen" are near each other, and the famous result $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ shows the geometry encodes meaning. Word2Vec and GloVe are the classic pre-trained embedding methods.

	TF-IDF	Word Embeddings
Represents	Word importance by frequency	Word meaning in vector space
Captures meaning?	No	Yes — similar words are close
Order/context?	No	Partially (contextual embeddings, yes)
Best for	Strong, fast baseline	Semantic tasks, deep learning input

2.4 Contextual Embeddings

Word2Vec gives each word one fixed vector, but words are ambiguous — "bank" (river vs. money) should differ by context. Contextual embeddings from transformers (Week 7: BERT) produce a different vector for a word depending on its sentence, which is why transformer-based models outperform older approaches on nuanced language tasks.

Hands-On Lab: Vectorizing Text

- Step 1: Apply TF-IDF to the cleaned text from Day 1 and train a simple classifier as a text baseline.
- Step 2: Load pre-trained word embeddings and find the nearest neighbors of a few words to see the semantic geometry.
- Step 3: If the project is text-based, compare a TF-IDF model against the Week 7 LSTM/transformer on the same metric.
- Step 4: Document which representation fits the project and why, in Markdown.

Tools Used

Scikit-learn (TfidfVectorizer) • Gensim / pre-trained embeddings • Jupyter/Colab

Day 3 — Computer Vision Preprocessing with OpenCV — 8 hours

Learning Objectives

- Preprocess images with OpenCV: resize, color-convert, normalize.
- Build an augmentation pipeline for image training data.
- Match preprocessing to a pre-trained model's expectations.

Key Topics

- Why images need standardization
- OpenCV: read, resize, color conversion, normalization
- The BGR vs. RGB pitfall

- Building an augmentation pipeline
- Matching preprocessing to transfer-learning models

Lesson Content

3.1 Why Images Need Preprocessing

Like text, raw images must be standardized before a model can use them: different sizes, color formats, and pixel ranges will break or degrade training. Computer vision preprocessing makes every image consistent — the same size, the same channel format, the same normalized pixel range — so the model sees uniform input.

3.2 OpenCV Fundamentals

OpenCV is the standard image-processing library. The core operations every CV pipeline uses:

Operation	Purpose
Read / resize	Load an image and standardize its dimensions
Color conversion	Switch between BGR, RGB, and grayscale as needed
Normalization	Scale pixel values from 0-255 to 0-1 for stable training
Edge detection	Highlight boundaries (e.g. Canny) for classical feature work

```
import cv2
img = cv2.imread("image.jpg")
img = cv2.resize(img, (128, 128))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = img / 255.0 # normalize pixel values to 0-1
```

OpenCV reads images in BGR order, not RGB. Forgetting to convert is a classic bug that silently corrupts colors — always convert to RGB before feeding a model trained on RGB data.

3.3 Building an Augmentation Pipeline

Data augmentation (introduced in Week 7) is applied systematically here as a preprocessing pipeline: random flips, rotations, zooms, brightness shifts, and crops generate varied training examples on the fly. This expands a small dataset and forces the model to generalize rather than memorize — the primary defense against overfitting in the Image Classifier project.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=20, zoom_range=0.15,
    horizontal_flip=True, brightness_range=[0.8, 1.2],
    rescale=1./255)
```

3.4 Connecting to Transfer Learning

Preprocessing must match what a pre-trained model expects. When using transfer learning (Week 7), the input images must be resized and normalized exactly the way the base model (ResNet, MobileNet) was originally trained — most frameworks provide a matching `preprocess_input` function for this. Mismatched preprocessing is a common cause of poor transfer-learning results.

Hands-On Lab: Image Preprocessing + Mentor Review

- Step 1: Build an OpenCV preprocessing function: read, resize to a fixed size, convert to RGB, normalize.
- Step 2: Create an augmentation pipeline and visualize several augmented versions of one image.
- Step 3: If the project is image-based, apply the correct `preprocess_input` for the chosen pre-trained model.

- Step 4: Open a pull request with the integrated notebook for the mid-sprint Mentor Code & Notebook Review and address the feedback.

Tools Used

OpenCV • TensorFlow/Keras (ImageDataGenerator) • Matplotlib • Git & GitHub (pull request)

Day 4 — Model Integration & Error Analysis — 8 hours

Learning Objectives

- Integrate preprocessing and the model into one end-to-end pipeline.
- Guarantee training/serving consistency to avoid skew.
- Perform error analysis with the confusion matrix and misclassified examples.

Key Topics

- What integration means
- The end-to-end predict pipeline
- Training/serving skew and how to prevent it
- Error analysis with the confusion matrix
- Inspecting misclassified examples

Lesson Content

4.1 What Integration Means

So far the project exists as separate notebook cells: cleaning here, a model there, evaluation elsewhere. Integration assembles these into one coherent, ordered pipeline — raw input enters one end, a prediction comes out the other — with every step applied consistently. This is what turns experimental code into something deployable in Sprint 4.

4.2 The End-to-End Pipeline

The integrated pipeline chains preprocessing and the model into a single object, exactly like the Scikit-learn Pipeline from Week 4, extended to text or images. For classical/tabular projects this is a literal sklearn Pipeline; for deep learning it is a clean sequence of preprocessing functions feeding the trained model. Either way, one function should take raw input and return a prediction.

```
def predict(raw_input):
    cleaned = preprocess(raw_input)      # same steps as training
    vectorized = vectorizer.transform(cleaned)
    prediction = model.predict(vectorized)
    return prediction
```

The golden rule of integration: preprocessing at prediction time must be identical to preprocessing at training time. Any mismatch (different scaling, different tokenization) silently corrupts predictions — this is training/serving skew, a top real-world ML bug.

4.3 Error Analysis

A single accuracy number hides where a model fails. Error analysis examines the specific cases the model got wrong to find patterns: Does it fail on one class more than others? On short inputs? On a particular category? For classification, the confusion matrix (Week 3) shows exactly which classes are confused with which. These patterns point directly to what to fix — better features, more data for a weak class, or a different threshold.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
cm = confusion_matrix(y_test, predictions)
sns.heatmap(cm, annot=True, fmt="d")  # which classes get confused?
```

4.4 Inspecting Misclassified Examples

Beyond the confusion matrix, pull out the actual examples the model misclassified and read them. A misclassified review that is genuinely ambiguous is a data-quality issue; a clear-cut case the model got wrong reveals a real model weakness. This qualitative step is what separates a superficial evaluation from a professional one.

Hands-On Lab: Integrating and Analyzing the Model

- Step 1: Wrap the project's preprocessing and model into a single `predict()` function taking raw input to a prediction.
- Step 2: Verify prediction-time preprocessing exactly matches training-time preprocessing.
- Step 3: Produce a confusion matrix and identify the model's most common error type.
- Step 4: Pull out at least three misclassified examples, read them, and categorize each as a data issue or a model weakness in Markdown.

Tools Used

Scikit-learn / TensorFlow • Seaborn • Pandas • Jupyter/Colab

Day 5 — Full Evaluation, Explainability & Sprint Review — 8 hours

Learning Objectives

- Evaluate the model fully with task-appropriate metrics against the baseline.
- Explain individual and global predictions with SHAP.
- Complete the full Sprint 3 review and retrospective cycle.

Key Topics

- Task-appropriate metrics (recap from Weeks 3-4)
- Handling imbalance: SMOTE, precision-recall trade-off
- Explainability with SHAP: global and per-prediction
- Communicating results to non-technical stakeholders
- Sprint Review and Retrospective

Lesson Content

5.1 Full, Task-Appropriate Evaluation

Sprint 3 closes with a complete evaluation, using the right metrics for the task (the choices from Weeks 3-4): regression uses RMSE/MAE/ R^2 ; classification uses precision, recall, F1, and AUC-ROC, never accuracy alone on imbalanced data. Every metric is reported against the Week 6 baseline so the model's value is unambiguous.

5.2 Handling Imbalance (for Fraud/Churn Projects)

For imbalanced tasks like the Fraud Detection project, evaluation must center on the minority class. Techniques from the outline apply here: SMOTE oversampling to balance training data, and reporting the precision-recall trade-off rather than accuracy. Choosing the operating threshold deliberately — based on the cost of a false negative vs. false positive — is part of a professional evaluation.

5.3 Explainability with SHAP

A model that predicts well but cannot be explained is hard to trust or deploy. SHAP (SHapley Additive exPlanations) quantifies how much each feature pushed a specific prediction up or down, giving both global feature importance and per-prediction explanations. This directly supports communicating results to non-technical stakeholders — a core program outcome — and is explicitly expected for the Fraud Detection project.

```
import shap
explainer = shap.Explainer(model, X_train)
```

```
shap_values = explainer(X_test)
shap.summary_plot(shap_values, X_test)    # global feature importance
shap.plots.waterfall(shap_values[0])    # why THIS prediction was made
```

5.4 Sprint Review & Retrospective

Sprint Review: demo the integrated pipeline and full evaluation to the mentor — metrics vs. baseline, error analysis, and SHAP explanations. Per the acceptance criteria: the notebook runs cleanly, code is on the correct branch, the pull request is approved, results are documented, and metrics are logged. The Retrospective sets one concrete change for Sprint 4, which is deployment and final polish. After Sprint 3, the model is complete and trustworthy — Sprint 4 makes it usable.

Hands-On Lab: Sprint 3 Close-Out

- Step 1: Produce the full evaluation for the project with the correct metrics, reported against the Week 6 baseline.
- Step 2: If the task is imbalanced, apply SMOTE or threshold tuning and report the precision-recall trade-off.
- Step 3: Generate SHAP global feature importance and at least one per-prediction explanation.
- Step 4: Ensure all Sprint 3 work is committed, the pull request is merged after mentor approval, and results are documented.
- Step 5: Present the Sprint Review, then write the Retrospective with one concrete change for Sprint 4 (deployment).

Tools Used

SHAP • Scikit-learn / imbalanced-learn (SMOTE) • Matplotlib / Seaborn • Git & GitHub

6. Week 8 Deliverables

By the end of Week 8 (Sprint 3), every intern must submit the following to their mentor and GitHub repository:

- The Sprint 3 plan and a text-cleaning pipeline (tokenize, clean, lemmatize) preserving task-critical signal.
- A text-representation notebook comparing TF-IDF and embeddings, with a chosen representation justified.
- An OpenCV preprocessing and augmentation pipeline matched to the project's model (where image-based).
- An integrated end-to-end predict() pipeline with verified training/serving consistency.
- A full evaluation: task-appropriate metrics vs. baseline, error analysis, and SHAP explanations.
- All Sprint 3 work committed to GitHub via reviewed pull requests, plus the written Sprint Retrospective.

7. Week 8 Evaluation Criteria

Scored by the assigned mentor at the Sprint 3 review using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
NLP / CV preprocessing	Basic preprocessing with issues	Correct, task-appropriate preprocessing	Clean, justified pipeline; preserves task-critical signal
Integration quality	Model runs but pipeline is fragmented	Clean end-to-end pipeline, consistent preprocessing	Robust pipeline; no training/serving skew, deploy-ready
Evaluation & error analysis	Single metric, no error analysis	Right metrics vs. baseline + confusion matrix	Rigorous; error analysis drives concrete improvements
Explainability & communication	No explainability	SHAP applied and interpreted	Clear explanations suitable for non-technical stakeholders
Sprint discipline & Git workflow	Misses steps, sporadic commits	Full sprint cycle, PR review, clear narrative	Proactive; consistent, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

8. Technical Stack — Week 8

NLP	NLTK / spaCy (tokenize, lemmatize), Scikit-learn (TfidfVectorizer), Gensim / GloVe embeddings
Computer Vision	OpenCV, TensorFlow/Keras (ImageDataGenerator), pre-trained models
Integration	Scikit-learn Pipeline / custom predict() function
Evaluation & Explainability	Scikit-learn metrics, imbalanced-learn (SMOTE), SHAP
Workflow	Jupyter/Colab, MLflow, Matplotlib/Seaborn, Git & GitHub

9. Notes & Best Practices

This sprint turns a trained model into a finished component. Two rules dominate: prediction-time preprocessing must exactly match training-time preprocessing (or predictions silently break), and evaluation must go beyond a single number — error analysis and SHAP reveal what the model actually learned. After Sprint 3 the model is complete and trustworthy; Sprint 4 makes it deployable and public.